Lập trình mạng · Chương 4

gRPC: Protocol Buffers và lời gọi unary

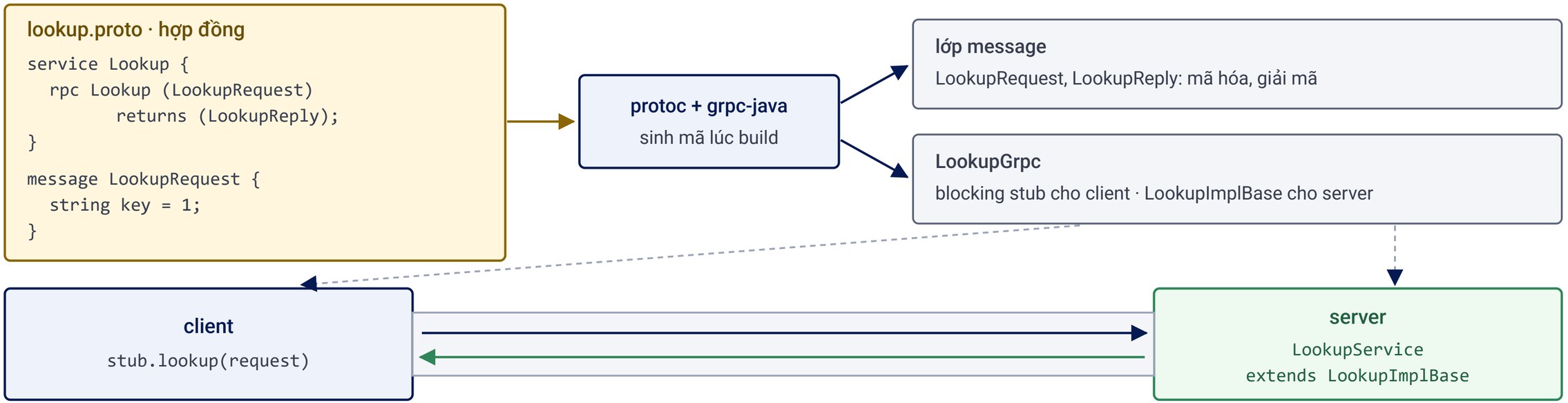
TS. Đặng Ngọc Hùng

gRPC: quay lại RPC với hợp đồng chặt và message nhị phân

1. REST + JSON: hợp đồng **không bắt buộc**; gõ sai tên trường chỉ lộ ra lúc chạy
2. JSON là văn bản, mỗi message mang cả **tên trường**
3. **gRPC**: hợp đồng bằng **Protocol Buffers**, message nhị phân, chạy trên **HTTP/2**

> **Câu hỏi của phần:** định nghĩa dịch vụ bằng file .proto, sinh mã và gọi một RPC thế nào?

Từ file .proto đến lời gọi: protoc sinh lớp message và stub



một kết nối HTTP/2, nhiều lời gọi song song (multiplexing)

Sinh mã: từ **file .proto**, protoc tạo lớp message và stub; client và server dùng chung một hợp đồng.

Protocol Buffers mã hóa bằng số trường: 7 byte so với 15 byte JSON

LookupRequest { key: "hanoi" }

0a	05	68 61 6e 6f 69
tag	độ dài 5	"hanoi" (UTF-8)

0a: trường số 1, kiểu chuỗi

Protocol Buffers: 7 byte

JSON cùng nội dung

```
{"key":"hanoi"}
```

tên trường "key" đi trên mạng

15 byte

kích thước đo được	Protocol Buffers	JSON
request	**7 byte**	15 byte
response	**31 byte**	52 byte

so sánh	REST + JSON	gRPC + Protocol Buffers
hợp đồng	không bắt buộc (OpenAPI)	bắt buộc: file .proto, sinh mã
giao thức	HTTP/1.1 hoặc HTTP/2	HTTP/2
trình duyệt	gọi trực tiếp	cần gRPC-Web

Protocol Buffers: mỗi trường gồm tag (số trường, kiểu), độ dài, dữ liệu; tên trường không đi trên mạng.

File .proto: dịch vụ, RPC unary và message có số trường

```protobuf
service Lookup {
  rpc Lookup (LookupRequest) returns (LookupReply);  // unary: một request, một response
}

message LookupRequest {
  string key = 1;  // số trường 1: định danh trường khi mã hóa, không được đổi
}

message LookupReply {
  bool found = 1;
  string value = 2;
}
```

- `service`, `rpc`: dịch vụ và lời gọi **unary**
- `message`: mỗi trường có kiểu, tên và **số trường**
- Không có khóa: `found` bằng false; trường không gán mang giá trị mặc định

Server: kế thừa lớp cơ sở sinh ra, trả kết quả qua StreamObserver

Java

```
static final class LookupService extends LookupGrpc.LookupImplBase {
    @Override
    public void lookup(LookupRequest request, StreamObserver<LookupReply> responseObserver) {
        String value = DATA.get(request.getKey());
        LookupReply reply = LookupReply.newBuilder()
                .setFound(value != null)
                .setValue(value != null ? value : "")
                .build();
        responseObserver.onNext(reply);  // unary: gửi đúng một response
        responseObserver.onCompleted();  // rồi kết thúc lời gọi
```

Kết quả chạy

```
gRPC server on port 9199
Lookup(key="hanoi") on grpc-default-executor-0
Lookup(key="atlantis") on grpc-default-executor-0
```

Client: ManagedChannel và blocking stub cho lời gọi unary

Java

```
ManagedChannel channel = ManagedChannelBuilder.forAddress("127.0.0.1", 9199)
        .usePlaintext()  // HTTP/2 không mã hóa (h2c), chỉ để chạy cục bộ
        .build();
LookupGrpc.LookupBlockingStub stub = LookupGrpc.newBlockingStub(channel);
LookupRequest request = LookupRequest.newBuilder().setKey(key).build();
LookupReply reply = stub.lookup(request);  // lời gọi unary qua blocking stub
```

Kết quả chạy

```
hanoi -> found=true, value="Thủ đô của Việt Nam"
atlantis -> found=false, value=""
```

Đổi số trường làm client cũ đọc sai; chỉ thêm trường với số mới

```java
byte[] renumbered = LookupReplyRenumbered.newBuilder()
        .setFound(true).setValue("Thủ đô của Việt Nam").build().toByteArray();
print("server đổi số trường", LookupReply.parseFrom(renumbered));  // client cũ đọc theo số trường cũ
byte[] extended = LookupReplyExtended.newBuilder()
        .setFound(true).setValue("Thủ đô của Việt Nam").setVersion(7).build().toByteArray();
print("server thêm trường 3", LookupReply.parseFrom(extended));
```

Kết quả chạy

```
server đổi số trường: found=false, value="", 2 unknown field(s)
server thêm trường 3: found=true, value="Thủ đô của Việt Nam", 1 unknown field(s)
```

- Đổi số trường: client cũ đọc **sai**, `found=false`, không có ngoại lệ
- Thêm trường với **số mới**: client cũ đọc đúng các trường cũ, bỏ qua trường lạ

Câu hỏi 1

Trong file .proto, khai báo string key = 1; Số 1 dùng để làm gì?

A. Quy định thứ tự in các trường khi in message ra màn hình.

B. Giới hạn trường key dài tối đa 1 byte.

C. Định danh trường trong dữ liệu mã hóa; tên key không đi trên mạng.

D. Số phiên bản của message LookupRequest.

Câu hỏi 1

Trong file .proto, khai báo string key = 1; Số 1 dùng để làm gì?

A. Quy định thứ tự in các trường khi in message ra màn hình.

B. Giới hạn trường key dài tối đa 1 byte.

C. Định danh trường trong dữ liệu mã hóa; tên key không đi trên mạng.

D. Số phiên bản của message LookupRequest.

Số trường được ghi vào tag của trường trong byte mã hóa; bên nhận dựa vào số này, không dựa vào tên, để biết byte thuộc trường nào.

Câu hỏi 2

gRPC chạy trên phiên bản HTTP nào, và điều đó mang lại gì cho nhiều lời gọi tới cùng một server?

A. HTTP/2: nhiều lời gọi chạy song song trên một kết nối nhờ multiplexing.

B. HTTP/1.1: mỗi lời gọi một kết nối riêng nên không phải chờ nhau.

C. HTTP/3: không bao giờ bị head-of-line blocking.

D. Không dùng HTTP: gRPC chạy thẳng trên UDP.

Câu hỏi 2

gRPC chạy trên phiên bản HTTP nào, và điều đó mang lại gì cho nhiều lời gọi tới cùng một server?

A. HTTP/2: nhiều lời gọi chạy song song trên một kết nối nhờ multiplexing.

B. HTTP/1.1: mỗi lời gọi một kết nối riêng nên không phải chờ nhau.

C. HTTP/3: không bao giờ bị head-of-line blocking.

D. Không dùng HTTP: gRPC chạy thẳng trên UDP.

gRPC dùng HTTP/2: mỗi lời gọi là một HTTP/2 stream, nhiều lời gọi chia nhau một kết nối TCP.

Tổng kết

1. **gRPC**: RPC trên HTTP/2; hợp đồng là **file .proto** viết bằng **Protocol Buffers**
2. **Sinh mã**: protoc tạo lớp message và stub; client gọi RPC **unary** qua **blocking stub**
3. Mã hóa theo số trường: message nhỏ hơn JSON; chỉ thêm trường mới, không đổi số trường đã dùng

Từ khóa cần nhớ gRPC · Protocol Buffers · file .proto · sinh mã · unary · blocking stub

Mở rộng

Chủ đề nâng cao

- Tương thích ngược và tương thích xuôi của Protocol Buffers
- gRPC-Web cho trình duyệt

Bài tập mở rộng

Thêm RPC Update(key, value) vào lookup.proto, sinh lại mã và viết test gọi Lookup sau Update.

Tiếp theo

gRPC: streaming, deadline và mã trạng thái